DOC-SMCTA-GOV-001 · VERSIÓN 2.2.0

Registro Oficial de Decisiones de Arquitectura (ADR-001 a ADR-007)

Constancia Normativa bajo Estándar ISO/IEC/IEEE 42010 y Formato M. Nygard para Gobernanza del Equipo

Proyecto / Sistema	**Comité Técnico de Arquitectura y Control de Decisiones**
Fase / Área Normativa	**06 — Gobernanza de Ingeniería y Blindaje Técnico Oficial**
Autoría y Gobernanza	**Equipo de Ingeniería y Arquitectura — Proyecto SMCTA**
Fecha de Emisión	**Noviembre 2026**

Resumen

Este documento consolida el registro inmutable de las siete decisiones arquitectónicas estructurales (*Architecture Decision Records*) que gobiernan el diseño, la tecnología y los mecanismos de seguridad del sistema **SMCTA v2**, acompañado de su matriz maestra de estado (*Decision Log*). Su propósito institucional es preservar la memoria técnica del equipo, evitar la reapertura cíclica de debates ya resueltos con evidencia empírica y establecer límites vinculantes para futuras extensiones del monorepo.

Índice General de Contenidos

Resumen

Matriz Maestra de Decisiones (*Decision Log*)

ADR-001: Transición de Market Maker Directo (v1) a Mercado P2P con Custodia Escrow Segregada (v2)

ADR-002: Adopción de Monorepo TypeScript Estricto y `decimal.js` frente a Go o Rust

ADR-003: Persistencia Hexagonal Dual (`PGlite` WASM y PostgreSQL 15 `SERIALIZABLE`)

ADR-004: Concurrencia de Triple Capa, Liberación Lua en Redis y Libro Mayor Append-Only

ADR-005: Protocolo de Redención Física mediante QR Dinámico `HMAC-SHA256` y `token_version`

ADR-006: Migración de Maquetas `BREW/DESK` desde Dart/Flutter (`vers/`) hacia React White-Label (`@smcta/web`)

ADR-007: Sustitución de Liquidación Monetaria P2P Abierta por Trading P2P de Productos (Estilo Steam), Créditos de Circuito Cerrado e Item Changer con Incentivo por Descontinuación (Estilo KeyDrop)

Matriz Maestra de Decisiones (*Decision Log*)

ID	Título de la Decisión	Estado	Fecha	Subsistemas Afectados	Verificación Asociada
ADR-001	Transición de *Market Maker* v1 a Mercado P2P con Custodia *Escrow* Segregada	**Aceptado**	2026-09-20	Dominio, Legal, Ledger	`QA-03` , `escrowService.ts`
ADR-002	Monorepo Unificado en TypeScript Estricto + `decimal.js` frente a Go / Rust	**Aceptado**	2026-10-05	`@smcta/shared` , `backend` , `web`	`CC ⩽ 5` , `MI = 87,08`

ID	Título de la Decisión	Estado	Fecha	Subsistemas Afectados	Verificación Asociada
ADR-003	Persistencia Hexagonal Dual (`PGlite` WASM + PostgreSQL 15 `SERIALIZABLE`)	**Aceptado**	2026-10-18	`db/database.ts` , `pglite.ts` , `pg.ts`	`40/40` Vitest, `8/8` E2E
ADR-004	Defensa de Concurrencia en Triple Capa y Libro Mayor *Append-Only*	**Aceptado**	2026-10-28	`lockManager.ts` , `schema.sql`	`QA-02` (50 compradores), `SQLSTATE 28000`
ADR-005	Protocolo QR Dinámico `HMAC-SHA256` con `QrSecret` Opaco y `token_version`	**Aceptado**	2026-10-28	`qrSecret.ts` , `qrToken.ts` , `tpvService.ts`	`qrToken.test.ts` , `QA-04`
ADR-006	Consolidación de Maquetas `BREW/DESK` desde Dart/Flutter (`vers/`) hacia React	**Aceptado**	2026-11-03	`@smcta/web` , `seed.ts`	`build:ui` , Tenant `cafeteria`
ADR-007	Permuta P2P de Productos (Estilo Steam), Créditos Cerrados e *Item Changer* con Bonus por Descontinuación (Estilo KeyDrop)	**Aceptado**	2026-11-06	`pricing.ts` , `p2pService` , `Marketplace`	`CR-003` , `RN-07` , `RN-08` , `UC-05`

ADR-001: Transición de Market Maker Directo (v1) a Mercado P2P con Custodia Escrow Segregada (v2)

- **Estado:** Aceptado y Vigente (Reemplaza diseño conceptual v1).
- **Contexto y Problema:** El borrador original proponía que la plataforma recomprara instantáneamente cupones no consumidos utilizando su propia tesorería. Ello exponía al operador a corridas de liquidez ante eventos cancelados o caídas abruptas de demanda, además de configurar un riesgo severo de encuadramiento regulatorio como intermediación financiera no autorizada o emisión de derivados sin prospecto aprobado.
- **Alternativas Evaluadas:**

1. *Mantener recompra directa con encaje fraccionario (v1):* Descartado por insolvencia matemática ante retiros correlacionados (> 30% de la emisión).
2. *Prohibir cualquier transferencia secundaria (modelo prepago cerrado):* Descartado por anular la propuesta de valor central del proyecto.

3. *Mercado secundario exclusivamente P2P con custodia Escrow segregada al 100% nominal y bandas Price Collar (v2):* Seleccionado.

- **Decisión:** SMCTA opera exclusivamente como proveedor tecnológico SaaS *White-Label* de emparejamiento entre pares. El 100% del valor nominal P_{nom} pagado en la emisión primaria permanece bloqueado en cuenta de garantía y solo se libera al comercio tras validación física en TPV (`REDEEM_RELEASE`) o al usuario tras expiración reglamentaria (`REFUND`). La formación de precios secundarios queda confinada al intervalo $[\varphi_{min}, \varphi_{max}]$ fijado por el comercio.
- **Consecuencias:**
- *Positivas:* Riesgo de balance propio exactamente igual a cero; aislamiento total del capital comercial frente a plusvalías o descuentos secundarios; ingresos predecibles vía *take-rates* porcentuales.
- *Negativas / Compromisos:* La liquidez secundaria deja de ser instantánea contra la casa y pasa a depender de la existencia de un comprador par dentro de la ventana habilitada antes del cierre del mercado (`closure_hours_before_event`).

ADR-002: Adopción de Monorepo TypeScript Estricto y `decimal.js` frente a Go o Rust

- **Estado:** Aceptado y Vigente.
- **Contexto y Problema:** El equipo debía seleccionar el lenguaje del backend transaccional evaluando tres candidatos principales: TypeScript (Node.js 20 + Fastify 5), Go y Rust, considerando que el frontend *White-Label* y la suite E2E se implementarían en TypeScript/React/Playwright por un equipo distribuido en tres roles.
- **Factores Técnicos Evaluados:**

1. *Naturaleza de la carga:* El sistema está dominado por latencia de E/S en red y disco (`PostgreSQL SERIALIZABLE` + `Redis`), donde el tiempo en CPU del lenguaje representa menos del 2% del tiempo de respuesta total (`~7 ms` por operación bajo estrés).
2. *Simetría matemática cliente-servidor:* La previsualización instantánea de comisiones y bandas en el *slider* de publicación P2P debe arrojar al centésimo exacto el mismo resultado que la validación transaccional del servidor.
3. *Expresividad estructural y control de complejidad:* Se requerían interfaces limpias para Puertos y Adaptadores, tipado nulo estricto y verificación automatizada de complejidad ciclomática $CC \leq 5$.

- **Decisión:** Adoptar **TypeScript 5.7** con configuración máxima de rigor (`strict` , `noUncheckedIndexedAccess` , `exactOptionalPropertyTypes`) en un monorepo de *NPM*

Workspaces, centralizando la aritmética decimal de base 10 (`decimal.js` con redondeo bancario `ROUND_HALF_EVEN`) y los tipos de dominio en `@smcta/shared` .

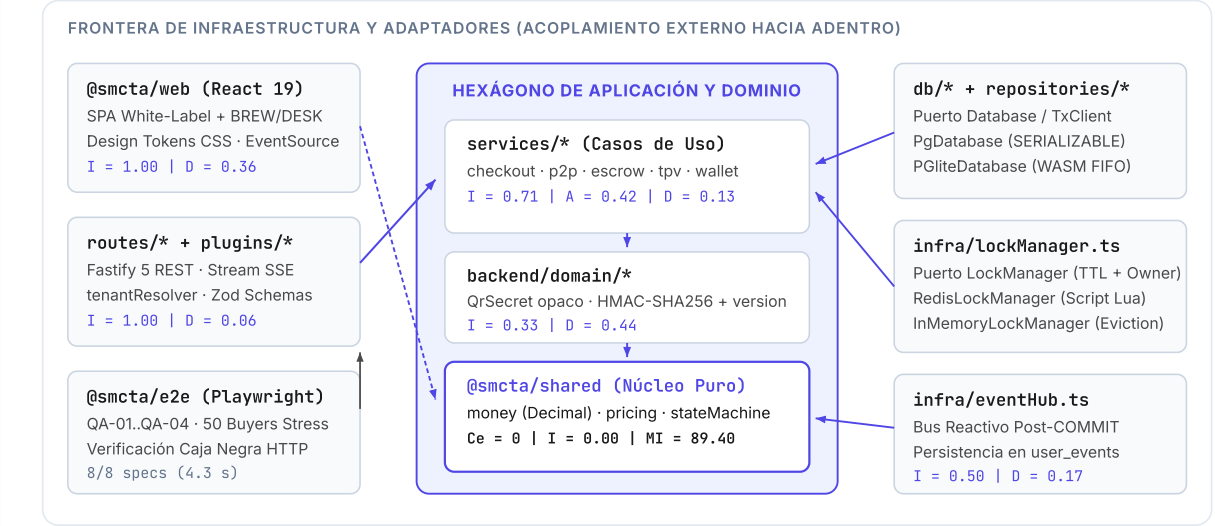


Figura 1. Topología de Arquitectura Hexagonal y dirección estricta de dependencias del monorepo SMCTA v2. Todas las flechas convergen hacia las abstracciones internas y el núcleo puro `@smcta/shared` (C_e = 0, I = 0,00).

- **Consecuencias:**
- *Positivas:* Cero divergencia de contratos DTO ni de redondeo financiero entre UI y API; velocidad superior de integración entre los Roles 1, 2 y 3; cumplimiento del 100% en complejidad ciclomática ($CC \leq 5$ verificado por AST sobre 593 funciones) y acoplamiento saludable ($D \leq 0{,}59 < 0{,}70$).
- *Negativas / Mitigación:* Debido a que el tipo primitivo `number` de JavaScript es coma flotante binaria IEEE 754, queda prohibido operar sumas o porcentajes monetarios fuera de las funciones exportadas por `@smcta/shared/src/money.ts` .

ADR-003: Persistencia Hexagonal Dual (`PGlite` WASM y PostgreSQL 15 `SERIALIZABLE`)

- **Estado:** Aceptado y Vigente.
- **Contexto y Problema:** Requerir una instancia Docker externa de PostgreSQL para correr pruebas unitarias o levantar demostraciones locales generaba fricción ambiental entre máquinas de distintos integrantes, mientras que sustituir PostgreSQL por SQLite en pruebas impedía verificar disparadores PL/pgSQL, tipos `ENUM` nativos e índices parciales reales.

- **Decisión:** Definir en `apps/backend/src/db/database.ts` el puerto abstracto `Database` (`query`, `withTransaction`) e implementar dos adaptadores intercambiables por configuración (`DATABASE_URL`):

1. `PgDatabase` (`pg.ts`): Adaptador productivo sobre `pg.Pool` que ejecuta cada unidad de trabajo bajo `BEGIN ISOLATION LEVEL SERIALIZABLE`.
2. `PGliteDatabase` (`pglite.ts`): Adaptador embebido sobre `@electric-sql/pglite` (motor PostgreSQL real compilado a WebAssembly) que ejecuta exactamente el mismo archivo `schema.sql` sin dependencias externas, serializando el acceso concurrente a su conexión única mediante una cola FIFO de promesas (`mutexChain`).

- **Consecuencias:** Paridad semántica del 100% en dialecto SQL, restricciones `CHECK`, índices únicos parciales y triggers entre desarrollo, integración continua y producción (`npm run dev` y `npm test` funcionan sin configuración manual).

ADR-004: Concurrencia de Triple Capa, Liberación Lua en Redis y Libro Mayor Append-Only

- **Estado:** Aceptado y Vigente.
- **Contexto y Problema:** En eventos de alta convocatoria, decenas de compradores presionan simultáneamente el botón de compra sobre la misma oferta secundaria. Un bloqueo ingenuo en memoria falla al escalar horizontalmente a múltiples réplicas del backend, un `DEL` directo en Redis puede borrar el candado de un tercero si el primer proceso sufrió una pausa, y una tabla contable ordinaria admite alteraciones accidentales vía `UPDATE`.
- **Decisión:** Implementar defensa en profundidad mediante tres capas ortogonales:

1. **Puerto `LockManager` con token de propietario:** `RedisLockManager` adquiere `SET key owner PX ttl NX` y ejecuta liberación condicional mediante script Lua (`if redis.call("get", KEYS[1]) == ARGV[1] then return redis.call("del", KEYS[1])`). `InMemoryLockManager` replica idéntica semántica de propietario con purga perezosa por TTL.
2. **Transacción `SERIALIZABLE` + `SELECT ... FOR UPDATE`:** Verificación pesimista de estado dentro del motor relacional.
3. **Invariantes físicos en `schema.sql`:** Índices parciales `uq_orders_open_coupon`, `uq_ledger_checkout_once`, `uq_ledger_terminal_once` y trigger `trg_escrow_ledgers_immutable` (`SQLSTATE 28000`).

- **Consecuencias:** Verificación determinista en `QA-02` (`50` peticiones paralelas resueltas en `358 ms`: `1` éxito `HTTP 200`, `49` rechazos limpios `423/409`, `0` inconsistencias).

ADR-005: Protocolo de Redención Física mediante QR Dinámico `HMAC-SHA256` y `token_version`

- **Estado:** Aceptado y Vigente.
- **Contexto y Problema:** Un código QR estático es vulnerable a duplicación fotográfica; un código puramente temporal (solo ventana de 30 s) sigue siendo vulnerable si el vendedor captura la imagen en $t = 0$ s, vende el cupón en $t = 5$ s y atraviesa el molinete en $t = 12$ s antes de que venza el intervalo temporal.
- **Decisión:**

1. Firmar cada token con `HMAC-SHA256` empleando el secreto exclusivo del *tenant* encapsulado en la clase opaca `QrSecret` (protegida contra serialización accidental en logs o JSON) y comparar firmas con `crypto.timingSafeEqual`.
2. Incluir dentro del payload firmado el contador monótono `coupons.token_version`, incrementado atómicamente en cada publicación, cancelación, compraventa P2P y canje.

- **Consecuencias:** Cualquier cambio de estado o titularidad invalida en `0 ms` todas las capturas o tokens emitidos previamente, aun cuando su ventana temporal de 30 segundos no haya expirado (`409 QR_STALE_TOKEN`).

ADR-006: Migración de Maquetas `BREW/DESK` desde Dart/Flutter (`vers/`) hacia React White-Label (`@smcta/web`)

- **Estado:** Aceptado y Vigente.
- **Contexto y Problema:** El equipo contaba con un prototipo visual de alta fidelidad en Dart/Flutter ubicado en `vers/` para el caso de uso de cafetería de especialidad (*BREW/DESK Specialty Coffee*), mientras que la arquitectura funcional multi-tenant conectada al backend operaba en React 19 (`apps/web`). Sostener dos pilas tecnológicas de UI duplicaba el mantenimiento y dejaba el diseño gastronómico desconectado de la contabilidad real.
- **Decisión:** Congelar y archivar `vers/` como especificación visual de referencia histórica, migrar fielmente la totalidad de sus vistas y componentes (portada oscura de barra, promociones cromáticas, modal de ficha técnica con curva temporal `24h / 7d / 30d` , catálogo filtrable `CartaPage` , carrito lateral conectado a `POST /api/v1/checkout/primary` y panel maestro-detalle `SettingsPage`) hacia `@smcta/web` ,

incorporar el *tenant* semilla `cafeteria` y depurar definitivamente los activos duplicados del repositorio (`9d621db`).

- **Consecuencias:** Unificación del 100% del frontend bajo un solo árbol de componentes tipados en React/TypeScript, conservando tanto la capacidad camaleónica *White-Label* para hoteles/festivales como la identidad visual exacta diseñada por el equipo para *BREW/DESK*.

ADR-007: Sustitución de Liquidación Monetaria P2P Abierta por Trading P2P de Productos (Estilo Steam), Créditos de Circuito Cerrado e Item Changer con Incentivo por Descontinuación (Estilo KeyDrop)

- **Estado:** Aceptado y Vigente (Refina `ADR-001` bajo `CR-003`).
- **Contexto y Problema:** Si bien `ADR-001` eliminó el riesgo de insolvencia del operador mediante órdenes P2P y custodia *Escrow*, liquidar compraventas secundarias en dinero fiduciario transferible entre cuentas bancarias de usuarios particulares imponía dos costos severos: (1) exigencia de integrar pasarelas reguladas con dispersión de fondos entre terceros (*split payments* sujetos a normativa PSP/BCRA, retenciones impositivas y KYC bancario), y (2) obligación de reembolsar efectivo cuando un comercio gastronómico descontinuaba un producto estacional o agotaba una partida específica. Paralelamente, un trueque puro 1 a 1 sin unidad de cuenta intermedia ("cambio mi combo de empanadas solo si otro usuario tiene exactamente un café comprado en su billetera") sufriría parálisis de liquidez por falta de doble coincidencia de necesidades y por asimetría de precios entre ítems del menú (`$4,90` vs. `$2,80`).
- **Alternativas Evaluadas:**

1. *Mantener liquidación P2P con retiro de dinero fiduciario a bancos externos:* Descartado por alta fricción regulatoria de intermediación de pagos y drenaje de efectivo ante bajas de catálogo.
2. *Trueque P2P puro 1 a 1 sin unidad de cuenta ni créditos:* Descartado por iliquidez severa e imposibilidad de compensar diferencias de valor entre productos de distinto precio nominal.
3. *Arquitectura híbrida de Permuta P2P de Productos (estilo Steam Market / Trade Offers) + Saldo de Créditos de Circuito Cerrado + Intercambiador de Equivalencias con Incentivo por Descontinuación (estilo KeyDrop Skin Changer):* Seleccionado.

- **Decisión:**

1. **Circuito Cerrado de Valor (*Closed-Loop Store Credits*):** El dinero fiduciario ingresa exclusivamente en la compra primaria. Dentro de cada *Tenant*, toda diferencia de valor

se liquida en **Créditos de Barra (`users.balance`)** equivalentes a `$1,00` nominal de consumo interno y **nunca retirables a cuentas bancarias externas**.

2. **Trading P2P de Productos entre Usuarios:** Un usuario puede catalogar su cupón activo (ej. *Sándwich + Latte* de `$4,20`) para intercambiarlo por otro producto específico de otro usuario (ej. *Espresso* de `$2,80`) más/menos una compensación en Créditos de Barra, o bien cederlo directamente contra Créditos de Barra dentro de los límites del *Price Collar* $[\varphi_{\min}, \varphi_{\max}]$. El cruce transfiere atómicamente la titularidad de ambos cupones e incrementa `token_version` en ambos activos.
3. **Motor *Item Changer* e Incentivo por Producto Descontinuado:** Cuando un producto es marcado como **Descontinuado (`is_discontinued = true`)** por el comercio, queda inhabilitado para el P2P entre usuarios y habilitado en el **Changer de Equivalencias** (interfaz de doble columna inspirada en KeyDrop). Al depositar el cupón descontinuado a la izquierda, el sistema le asigna un valor de rescate bonificado $V_{canje} = \text{round}_2(P_{nom} \cdot (1 + \frac{\beta_{disc}}{100}))$ (incentivo parametrizable β_{disc}, por defecto $+15\%$), permite elegir a la derecha uno o más productos equivalentes del catálogo activo (ej. un alfajor o un *Mokaccino*) y acredita el remanente positivo en el saldo de Créditos de Barra del usuario.

- **Consecuencias:**
- *Positivas:* Eliminación total de contingencias de intermediación bancaria/PSP entre consumidores; retención del 100% de la caja en el comercio ante productos descontinuados transformando un reembolso frustrante en una experiencia gamificada con premio ($+\beta_{disc}\%$); preservación íntegra de la infraestructura transaccional (`SERIALIZABLE`, `LockManager`, `decimal.js`, `Price Collar`, `token_version` y `escrow_ledgers`).
- *Negativas / Restricciones:* Los términos de servicio y la interfaz deben explicitar que los Créditos de Barra constituyen saldo promocional/prepago de consumo interno exclusivo en el local emisor, sin convertibilidad a moneda de curso legal.